Re-implementing a Cross-Modal Wearable Obstacle Detector: Reproducibility, Gaps and a Proposed Phone-Based Alternative

Allwin Raj R
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, Tamil Nadu, India

Madhu Sudharshanan
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, Tamil Nadu, India

Review 2 (DA2): base-paper re-implementation, cons and gap identification, and proposed work.

***Abstract*—We re-implement and evaluate the base paper of this project, WOAD (Gao et al., Nature Communications, 2025), a wearable obstacle-avoidance system whose phone runs a YOLOv5-based detector that fuses near-infrared and colour-mapped depth images with transformer blocks. The released scripts did not run with current libraries, so we wrote a standalone inference package whose outputs are bit-identical to the original code (maximum absolute difference 0.0) and showed by ablation that both modalities are used, with near-infrared dominant. A dataset audit found that the published data is not the data behind the reported detector accuracy (mAP@0.5 0.669): its classes differ and none of the run's 187 validation frames is in it. With the original metric code, the released checkpoint reaches a cross-dataset person AP@0.5 of 0.218 (validation, 129 people) and 0.173 (training split, 458 people), missing most cyclists and distant people, and it loses about 73 % of detections on darker sensor recordings. We derive eight gaps, among them the dependence on a specific sensor, unreproducible accuracy, the lack of motion semantics, distance and navigation, and propose a phone-camera system with a recall-selected detector, camera-motion-compensated tracking, offline speech and routing on a laptop GPU.**

***Index Terms*—assistive technology; visual impairment; electronic travel aid; obstacle detection; reproducibility; cross-modal learning; multi-object tracking; camera-motion compensation; offline speech; pedestrian navigation**

I. Introduction

A base paper is useful to a project only if its claims can be reproduced and its limits understood. This review therefore does three things for the base paper of the NeuroSonic project, *"A wearable obstacle avoidance device for visually impaired individuals with cross-modal learning"* (WOAD) by Gao et al., published in *Nature Communications* in March 2025 [1]:

1. it summarises the base system and the results the authors report;
2. it re-implements the published detector and evaluates it on the published data, to learn its strengths and its weaknesses ("cons") from evidence rather than from the paper alone; and
3. it identifies the gaps that prevent the base work from serving the users and settings of interest here (an ordinary smartphone, Indian streets, low cost, offline operation), and derives the proposed work from those gaps.

WOAD was chosen as the base paper because it is recent, published in a high-impact journal, addresses exactly the problem of obstacle avoidance while walking, reports strong user results, and releases its source code and data publicly. That last point is what makes a genuine re-implementation possible. All measurements in this review were made by us on one laptop (AMD Ryzen 7 7840HS, 15.3 GB RAM, NVIDIA RTX 4050 Laptop GPU) between 1 and 3 November 2026; numbers reported by the authors are labelled as such and kept separate.

II. The Base Paper: WOAD

A. System overview

WOAD addresses a stated tension: wearable obstacle-avoidance devices find it hard to be reliable, responsive, long-lasting and usable at the same time [1]. Its answer has four parts.

- **Sensing glasses.** Self-developed glasses of about 400 g, including an 80 g battery, carry a Synexens CS30 RGB-ToF module. Its near-infrared (NIR) camera and ToF depth sensor share a 100°×75° field of view at 640×480 pixels; depth is measured from 0.1 to 5 m.
- **Depth-aided compression.** To reduce transmission delay to the phone, a reinforcement-learning agent (discrete soft actor-critic [2]) uses obstacle status from consecutive depth frames to decide whether to send a frame and at what quality. The authors report a compression ratio above 95 %.
- **FPGA.** The compression policy runs on a custom 28 nm board with a multi-float-point vector streaming architecture; the authors report a latency of 0.095 ms and over 50 % lower power than their reference.
- **Cross-modal detector on the phone.** A Samsung Galaxy S23 Ultra runs a TensorFlow Lite version of a YOLOv5-based detector [3] with two input streams (NIR and colour-mapped depth) fused by transformer blocks [4]. Alerts are spoken at three hazard levels (attention, warning, danger), and three vibration motors on the forehead and temples indicate direction; the vibration frequency encodes distance (200 Hz below 3 m, 150 Hz above).

B. Detector architecture

Fig. 1 shows the detector as we verified it on the released checkpoint (run `exp72`). The NIR image passes through the first three YOLOv5s layers and the depth image through a separate stem of the same structure, each producing a 64×160×160 feature map. Both are scaled by a fixed 0.5 gate, average-pooled to 8×8, and flattened into 128 tokens of 64 dimensions with a learned positional embedding. Eight transformer blocks with eight attention heads let every NIR token attend to every depth token. The output tokens are

reshaped, upsampled, and used to re-weight each stream's features; the streams are concatenated, fused by a convolution, and passed through the standard YOLOv5s backbone, PAN neck and detection head, which outputs 25,200 candidate boxes for three classes (person, Ebike, Car). The checkpoint has 7,521,992 parameters (the paper reports 7,497,914 for its model [1]; the small difference may come from a different training run or from counting, and does not affect the analysis).

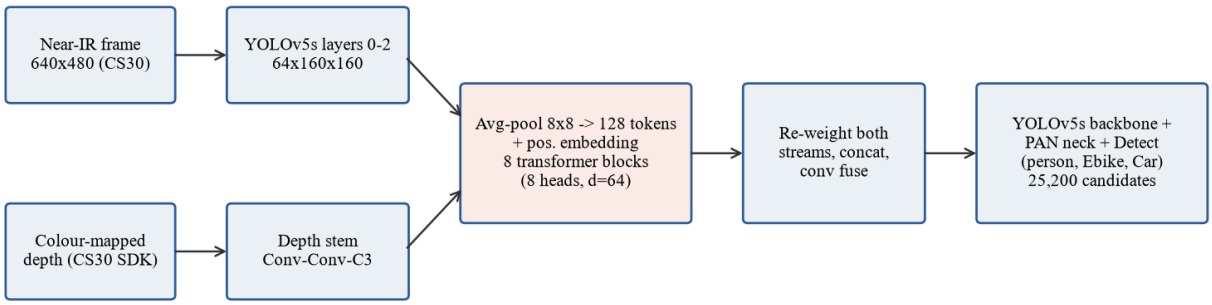


Fig. 1. WOAD cross-modal detector as verified on the released exp72 checkpoint: two input streams (near-infrared and colour-mapped depth), transformer fusion of 128 tokens, and a YOLOv5s detection network.

C. Results reported by the authors

TABLE I
RESULTS REPORTED IN THE BASE PAPER [1] (NOT MEASURED BY US)

Measure	Reported value
Collision avoidance, all scenarios	100 %
White cane, sudden indoor obstacles / fast cars at night	≈ 26 % / ≈ 40 %
End-to-end delay	< 320 ms
Operating duration; power	≈ 11 h; ≤ 4 W
System Usability Scale (WOAD vs cane)	83.75 ± 7.79 vs 80 ± 22.36
Participants	20 (12 quantitative over seven months, 8 survey)
Detector mAP@0.5 (training plot of the released run)	0.669 (person 0.771, Ebike 0.839, Car 0.395)

The authors also note limitations: occasional false alerts from shadows or snowflakes, concern that the glasses attract attention, and the need for a training period [1]. The detector accuracy in the last row of Table I is not stated in the main text of the paper; it is read from the precision–recall plot saved with the released training run, and it is the figure we attempted to reproduce.

III. RE-IMPLEMENTATION OF THE BASE PAPER

A. What was released, and its state

The public repository contains five parts: data collection with the CS30 SDK, the reinforcement-learning compression, the FPGA design, the smartphone deployment, and the cross-modal detector with its training run `exp72`. Code and data are distributed through nine Google Drive links. We inspected the detector code, the checkpoint and the published dataset in detail and, for the other links, read the archive indexes without downloading the whole archives.

The detector scripts did not run with current libraries. We found:

- the training, validation and detection scripts call the model with one concatenated input, whereas its forward function expects two separate tensors (NIR and depth);
- use of NumPy aliases that have been removed (`np.int`) and of PyTorch loading defaults that have changed;
- a missing `clouddir` argument in one code path, and a detection script that exits before non-maximum suppression;
- no saved results file and an empty TensorBoard log for the reported run; only the training plots remain.

Modifying the original repository would have made comparison impossible, so it was left untouched and a separate, standalone inference package was written.

B. Faithful re-implementation

The package (`woad/`) has three parts. A *loader* reads the pickled checkpoint by temporarily mapping the original module names to vendored copies of the nine classes the checkpoint needs. It checks the file's SHA-256 hash and the module tree, and it refuses a second, incompatible five-class checkpoint found at the repository root. A *preprocessing* module reproduces the original validation pipeline: the NIR image is resized so that its long side is 640 pixels (area interpolation) and letter-boxed to 640×640; the depth image is median-filtered (3×3) and letter-boxed; both are converted to RGB channel-first tensors in [0, 1]. The *detector* applies the original non-maximum suppression and coordinate scaling, so its detections are in the original image coordinates.

Equivalence test. A tool runs the original model code in a separate process and our re-implementation on the same seeded inputs and compares the raw outputs. The maximum absolute difference is **0.0**: the re-implementation is bit-identical to the original.

Modality ablation. To confirm that both inputs matter, we measured the total detection confidence when inputs are altered. Swapping NIR and depth leaves 8 % of the confidence; blanking the NIR input removes all detections; blanking the depth input leaves 65 %. Both modalities are used, and NIR dominates. Automated tests encode these properties, and deliberately breaking the preprocessing makes them fail.

C. Dataset audit

The detector's published data (`VOCdetiv_data.rar`, 1.31 GB) was downloaded, its size checked against the server and its RAR checksums verified. It holds four nested variants with 4 to 10 classes; the largest (nc_10) has 1,286 training and 365 validation image pairs with classes such as person, box, bike, zebra crossing, telegraph pole, doors, building entrance and traffic signs. The audit found that **this is not the dataset behind the reported run**:

- the checkpoint's classes are person, Ebike and Car; the dataset has no Ebike or Car class;
- none of the run's 187 validation frames (recovered from the run's plots) appears among the dataset's 1,651 unique images: the best normalised-correlation similarity was 0.72, while the same matcher recovers deliberately degraded dataset frames at above 0.9;
- dates and file paths in the archive differ from those of the run.

We also checked the other Drive links. The code bundle, two unlabelled recordings and the SDK do not contain the labelled person/Ebike/Car data either; one link returns HTTP 404. Other data issues: one orphan label, 53 file names reused between training and validation with different pixels, and one class ("Down stairs") without instances.

D. Evaluation method

The only class shared by model and data is *person*, so the evaluation is a cross-dataset person measurement. The evaluation tool reproduces the original validation settings exactly (Table II). Its AP code was verified to give identical values to the original code on random data.

TABLE II
EVALUATION SETTINGS (IDENTICAL TO THE ORIGINAL VALIDATION CODE)

Item	Setting
Inputs	NIR (`images/`) + depth (`clouds/`), same file name
Input size	640, letter-boxed
NMS	confidence 0.001, IoU 0.6, max. 300 detections
Matching	IoU 0.50:0.05:0.95
AP	101-point interpolation (a perfect detector scores 0.995)
Classes	dataset 0 (person) → model person; other classes ignored
Precision	FP32 on CPU (original: FP16 on CUDA)

E. Results

TABLE III
MEASURED PERSON ACCURACY OF THE WOAD EXP72 CHECKPOINT ON THE PUBLISHED DATA (NC_10)

Split	Images	People	P	R	AP@0.5	AP@.5:.95
val	365	129	0.345	0.310	**0.218**	0.073
train	1,286	458	0.361	0.199	**0.173**	0.050

Table III gives the results; precision (P) and recall (R) are at the maximum-F1 operating point (confidence 0.18 on val, 0.33 on train). Fig. 2 shows the precision–recall curves. At the original plotting settings (confidence 0.25, IoU 0.45) the model detected 37 of 129 people in the validation split and missed 89, with 5 unmatched detections; in the training split it detected 104 and missed 345. The forward pass took about 126–131 ms per image on the CPU in batches of 8.

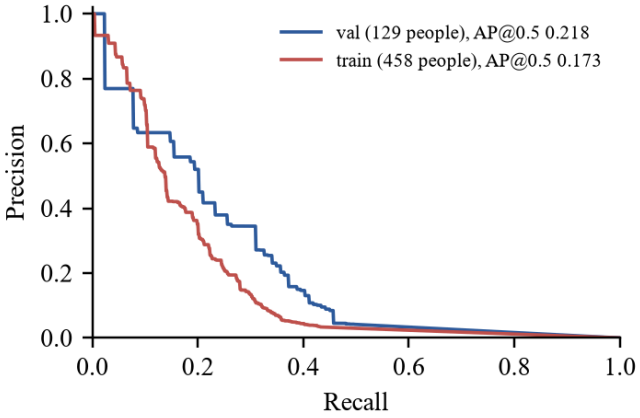


Fig. 2. Person precision–recall curves (IoU 0.5) of the WOAD checkpoint on the published dataset, measured with the original metric code.

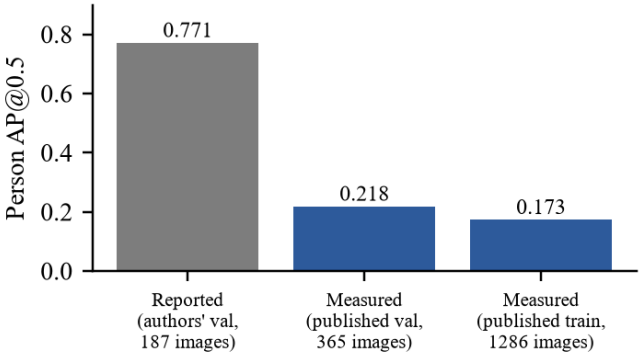


Fig. 3. Person AP@0.5: value read from the authors' training plot (their own validation split) versus our cross-dataset measurements. The datasets differ, so this is *not* a like-for-like comparison; it shows that the reported value could not be reproduced with the published data.

Interpretation. Fig. 3 contrasts the reported person AP (0.771) with our measurement (0.218 val, 0.173 train). Because the datasets differ, the gap measures *generalisation* to new scenes, not an error in the paper. The model is fairly precise but misses most people in these scenes. Visual inspection of sampled frames showed that close, upright people are detected with high confidence (0.74–0.87), whereas cyclists and distant people indoors are usually missed. The bicycle class of the dataset scored essentially zero as "Ebike", which is expected given the different class definitions and is not reported as an accuracy figure.

F. Brightness and live-input experiment

A raw CS30 recording released with the repository is much darker than the training-like data (mean NIR brightness 59 versus about 140). Darkening training-like frames to that level removed about 73 % of detections, and histogram correction of the dark recording produced one correct detection and 56 false alarms. The model is therefore sensitive to sensor exposure, which motivated an input-quality gate in our implementation that flags frames that are too dark, too low in contrast, or almost without valid depth, and withholds alerts from unusable frames.

G. Re-use of the base model in this project

The re-implemented detector is kept in the project as a research path: a command-line tool and a "replay" mode run it on recorded NIR + depth sequences, with an alert engine that

requires confirmation over several frames, applies cooldowns and states the object position (left, ahead, right). Thirty-eight original-equivalence and model tests remain part of the automated suite.

IV. Reproduction Procedure and Repository Audit

A. Components of the released work

TABLE IV
Components of the base paper's public release and their state when received

Component	Content	State when received	Used here
Cross-modal detector	YOLOv5s-based two-stream network with transformer fusion; run exp72; classes person, Ebike, Car	checkpoint present; train/val/detect scripts broken with current libraries	yes: re-implemented for inference
Reported accuracy	mAP@0.5 0.669 in a training plot	no results file; empty TensorBoard log	attempted reproduction
Published dataset	`VOCdetiv_data.rar`, NIR + depth, nc_4 to nc_10	intact, but a different dataset from exp72	yes: cross-dataset evaluation
Data collection	CS30 SDK binaries, capture program	no capture source code	only the colour-map finding
RL compression	D-SAC training scripts	data link returns HTTP 404	no
FPGA	Vivado project for the policy network	not inspected in depth	no
Smartphone	Android application	inspected by archive index; per the project's audit it ships a stock COCO TFLite model	no

B. Procedure

1. **Isolate the original.** The original repository was copied read-only; no file in it was changed, so every comparison is against the authors' code as published.
2. **Identify the checkpoint.** The exp72 checkpoint was hashed (SHA-256 prefix `da043ff2`), its module tree listed and its dimensions read: eight transformer blocks, eight heads, model width 64, positional embedding 1×128×64, 7,521,992 parameters.
3. **Re-implement inference.** Loader, preprocessing, non-maximum suppression and coordinate scaling were written as a standalone package with the original numerical behaviour.
4. **Prove equivalence.** The original model code and the re-implementation were run in separate processes on identical seeded inputs; the outputs were identical.
5. **Test that the inputs matter.** Input swapping and blanking (ablation) and deliberate preprocessing faults were added as automated tests.
6. **Audit the data.** The published archive was downloaded and verified; classes, counts, duplicates and frame identity against the run's validation set were checked; the other download links were inspected through their archive indexes.
7. **Evaluate.** Person AP was computed with the original matching and AP code, verified to be numerically identical to the original on random data.
8. **Probe the live-input path.** A raw sensor recording was compared with the training-like data, and the effect of brightness on detections was measured.

C. Download links

Nine Google Drive links are listed in the repository. File names, sizes and dates were read with HTTP HEAD requests, and archive contents from each RAR5 archive's index with a few small HTTP range requests (validated by listing the downloaded dataset and comparing all 9,370 file sizes). The code bundle (1.70 GB) contains the YOLOv5 code, a third checkpoint of a different size, and the same images with labels only inside training caches. Two archives hold unlabelled NIR + depth sequences (600 and 501 frames), which are useful for qualitative replay. One archive holds the SDK binaries, others hold Android build files, the RL project and the FPGA files. One link is dead (HTTP 404). No link contains the labelled person/Ebike/Car data of the reported run.

V. Threats to Validity of the Re-implementation

- **Numerical precision.** We evaluated in FP32 on the CPU; the original validation used FP16 on CUDA. The effect is expected to be negligible but was not measured.
- **Different data.** The measured AP is cross-dataset. It shows weak generalisation to the published scenes, not an error in the authors' own validation. Whether some published images were in the run's *training* data cannot be checked; the low scores make large overlap unlikely.
- **Metric quirks of the original code.** A perfect detector scores 0.995, not 1.0, with the original AP interpolation; and in the original confusion analysis unmatched detections are counted only in images that also contain a match, so false positives are under-stated. We kept these behaviours deliberately, to stay comparable.
- **Class mapping.** Only person is semantically shared. Bicycles scored as "Ebike" (AP ≈ 0) are reported only as an exploratory observation.
- **Label anomalies.** One orphan label and 53 file names reused across splits with different pixels; no image was excluded, following the original label rules.

VI. Building on the Base Model: Alert Engine and Input-Quality Gate

To turn per-frame detections of the base model into spoken alerts for recorded sensor data, an alert engine and an input-quality gate were implemented around it. They are retained in the final system as its research "replay" path.

TABLE V
ALERT-ENGINE AND QUALITY-GATE PARAMETERS (DEFAULTS)

Parameter	Value
Alert confidence threshold	0.40
Confirmation: hits within window	2 of 3 frames
Track association IoU	0.30
Cool-down / repeat interval	5 s / 15 s
Max. object alerts per frame	2
Position bands (left / ahead / right)	<1/3, 1/3–2/3, >2/3
Class priority	Car > E-bike > person
Quality "warn": NIR mean / std below	100 / 40
Quality "unusable": NIR mean / std below	40 / 10
Quality "unusable": invalid depth above	95 %

The engine confirms an object when it is seen in two of three consecutive frames, groups objects of the same class and position into one message (e.g. "Person detected ahead."), orders messages by class priority and box size, applies cool-downs, and announces when detection becomes unavailable or resumes. The quality gate compares each frame's NIR brightness and contrast and the fraction of invalid depth pixels with thresholds derived from the brightness experiment, and it withholds object alerts from "unusable" frames instead of speaking possibly meaningless detections. Both components are covered by automated tests. Their design principles (temporal confirmation, position words instead of distances, explicit "unavailable" messages) carried over into the phone-camera system.

VII. CONS OF THE BASE WORK AND GAP IDENTIFICATION

The strengths of WOAD are real: an integrated hardware–software system, a principled two-modality detector, energy-aware compression, and an unusually thorough user study [1]. The re-implementation also exposed weaknesses. We separate *cons* (properties of the base work) from *gaps* (what is missing for the target use: an ordinary phone, Indian streets, low cost, offline operation) and map each gap to a contribution.

A. Cons observed

1. **Hardware lock-in.** The detector requires NIR images from the CS30 sensor and depth colour-mapped by the vendor SDK's own look-up table (not a standard colour map, and with undocumented range settings). An ordinary RGB camera cannot drive it: feeding RGB as "NIR" with a fake depth image is out of distribution, and our ablation shows that NIR carries most of the signal.
2. **Reproducibility.** The reported detector accuracy cannot be re-measured with the published data, the scripts did not run with current libraries, and the run's results files are missing.
3. **Generalisation.** On the published scenes, person AP@0.5 is 0.17–0.22; cyclists and distant people are mostly missed. For a warning system, low recall on people is a safety-relevant weakness.
4. **Exposure sensitivity.** Darker sensor recordings remove most detections.
5. **Narrow class set.** Three classes (person, Ebike, Car) in the released model.
6. **No distance in the released detector code.** The paper's device encodes distance in vibration, but the released detector outputs only class, box and confidence; distance would need the raw 16-bit depth, which the colourised image does not preserve.
7. **Frame-by-frame semantics.** The released detector code has no tracking, so it cannot distinguish an object moving into the path from the apparent motion caused by the wearer's own walking.
8. **Obstacle-only.** There is no route guidance or speech-command interface in the released code.

TABLE VI
GAP IDENTIFICATION AND THE CONTRIBUTION THAT ADDRESSES EACH GAP

#	Gap	Evidence	Consequence	Proposed contribution
G1	Needs a specific NIR + ToF sensor and custom glasses	model inputs; ablation (blank NIR → 0 detections)	unusable with a phone; costly	phone RGB camera as input; detector chosen for RGB
G2	Reported accuracy not reproducible	dataset audit; missing run files	performance unknown	controlled benchmark of 4 detectors on the same annotated images
G3	Low person recall on new scenes	AP@0.5 0.218 / 0.173	missed warnings	select by recall; temporal confirmation absorbs false positives
G4	No motion semantics	frame-wise code	cannot warn "moving into path" reliably	camera-motion-compensated tracking; entering and crossing events
G5	No distance in released code; risk of over-claiming	outputs: class, box, score	misleading messages possible	no distance or "safe" statements; walking-path states only
G6	No navigation or voice interface	released code	user needs other apps	offline map, A* routing, spoken turns, offline voice commands
G7	Dependence on custom FPGA and compression for latency	system design	hard to replicate	off-the-shelf laptop GPU over local Wi-Fi; latency measured at the real frame rate
G8	Connectivity and privacy	cloud alternatives need internet	unreliable outdoors	all local functions offline; images never stored; cloud description opt-in

Gaps G1 and G2 are decisive: they mean that the base model cannot be the live detector of a phone-based system. The proposed work therefore keeps WOAD as a verified research baseline and builds a new perception path for the phone camera, while addressing G3–G8 in the system design.

VIII. PROPOSED WORK

A. Problem statement

Provide a pedestrian who carries an ordinary smartphone with short, timely spoken information about recognised objects in the walking path, including objects moving into or across it, together with spoken route guidance; without

depending on the internet, without storing camera images, without special sensors, and without overstating what the system can see.

B. Objectives

1. **O1 (real time):** process phone frames at about 5 frames/s, with an answer well within one frame interval (200 ms).
2. **O2 (detector):** choose the detector by measured accuracy, recall, latency and licence, with a CPU fall-back.
3. **O3 (motion):** announce people or vehicles *entering* or *crossing* the walking path, but not the apparent motion caused by the user's own walking.
4. **O4 (offline):** keep detection, speech output, voice commands and routing working without internet; make cloud description opt-in.
5. **O5 (honesty):** never state distances or safety; say "appears clear", never "safe".
6. **O6 (verification):** verify each function with automated tests, and state what is not verified.

C. Proposed architecture

The phone becomes a thin client running in Safari. It captures camera frames, microphone audio and GPS, and plays speech. A laptop with a consumer GPU on the same Wi-Fi network runs all engines (Fig. 4). This replaces the custom glasses, FPGA and compression of the base system (G1, G7) with hardware the user is likely to have or can borrow, at the cost of depending on the laptop.

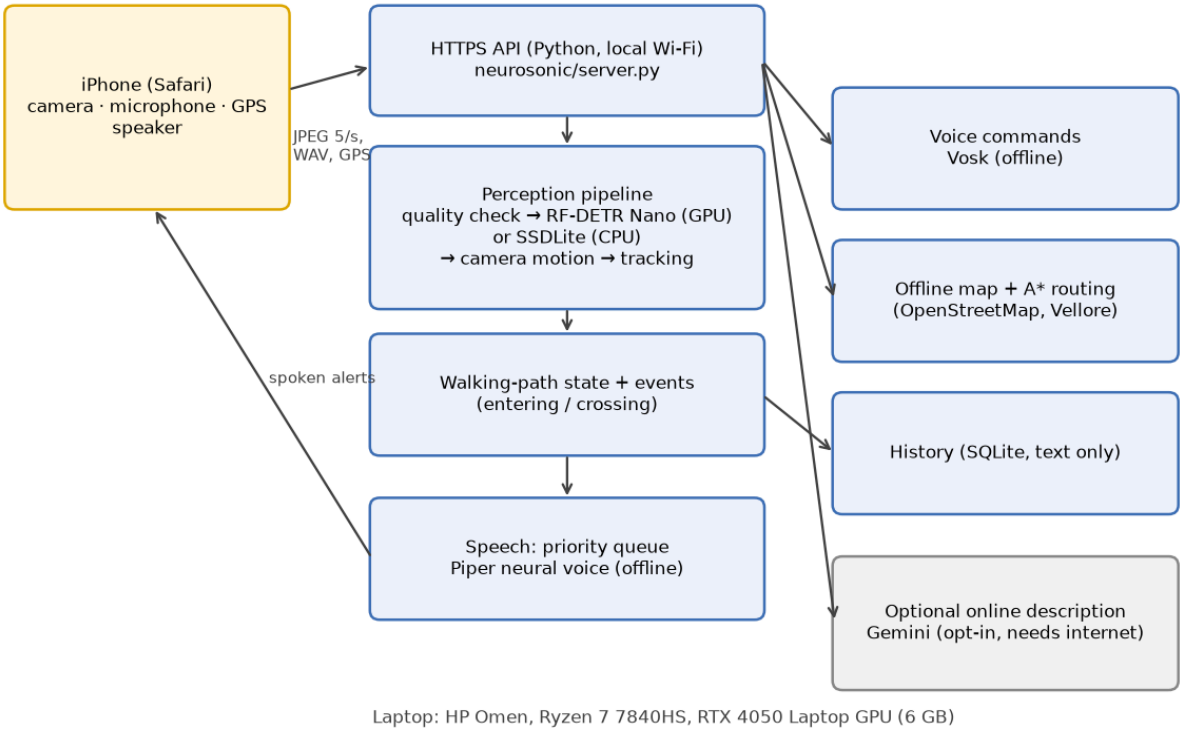


Fig. 4. Proposed architecture. The phone (browser) sends frames, audio and position over HTTPS on the local network; the laptop runs perception, events, neural speech, offline voice commands, offline map routing and text-only history. Only the optional scene description uses the internet, after the user opts in.

D. Methodology

- **Detector selection (G1–G3).** Benchmark candidate RGB detectors trained on COCO [5], from the established SSDLite/MobileNetV3 [6], [7] to the YOLO family [8] and the real-time detection transformer RF-DETR [9], [10], on the same annotated person images with the same AP code used for WOAD; select by recall first, because a missed person is worse than an extra caution.
- **Camera motion and tracking (G4).** Estimate the camera's own motion between frames from background corner points [11] tracked with pyramidal Lucas–Kanade flow [12], [13] and a RANSAC similarity fit [14]; predict each track's box with that motion and associate detections in two stages as in ByteTrack [15], [16]; store each object's motion *relative to the background*.
- **Walking-path states and events (G4, G5).** Define a lower-central walking-path region; confirm path states over 2–3 frames; raise "entering" and "crossing" events only for confirmed tracks with sufficient background-relative speed, and suppress all events during strong camera motion.
- **Speech, commands and routing (G6, G8).** Offline neural speech with Piper [17], [18], offline command recognition with Vosk [19], and A* routing [20] on an OpenStreetMap extract [21] of Vellore, with spoken turn instructions.
- **Serving at the real frame rate (G7).** Measure latency at 5 frames/s, not only back to back, and correct any serving bottleneck found.

E. Evaluation plan

TABLE VII
EVALUATION PLAN FOR THE PROPOSED WORK

Question	Measure	Where
Which detector?	person AP@0.5, recall @0.40, GPU/CPU latency, licence	laptop benchmark
Does FP16 change accuracy?	AP and latency FP32 vs FP16	laptop benchmark
Real-time?	frames/s, capture→answer time	browser test, fake camera
Stable over time?	errors, latency, memory over 10 min	soak test
Correct events, no false alarms from panning?	unit tests with synthetic camera motion	automated
Offline?	functions with network blocked	automated, browser
Works on iPhone, Wi-Fi, streets, users?	23-scenario protocol, 5 trials each	field (pending)

F. Differentiation from the base paper

TABLE VIII
BASE PAPER VERSUS PROPOSED WORK

Aspect	WOAD (base)	Proposed (NeuroSonic)
Sensor	NIR + ToF glasses (CS30)	phone RGB camera
Compute	FPGA + phone	laptop GPU over local Wi-Fi
Detector	YOLOv5 + transformer fusion, 3 classes	RF-DETR Nano, 23 classes, CPU fall-back
Temporal logic	per frame	motion-compensated tracking, events
Distance	from depth (device)	none stated (honest by design)
Navigation	none	offline map, A*, spoken turns
Voice input	none	offline commands
Cost to user	custom device	existing phone + laptop
User study	20 participants	not yet (protocol prepared)

IX. CONCLUSION

The base paper presents a well-engineered wearable with strong user results [1]. Re-implementing its detector showed that the released model runs bit-identically to the original code once a standalone loader and preprocessing are written, and that it genuinely uses both modalities. It also showed that the reported detector accuracy cannot be reproduced with the published data, that on the published scenes the model misses most people (person AP@0.5 0.218 and 0.173), that it is sensitive to sensor exposure, and that it cannot be driven by an ordinary phone camera.

These findings define the proposed work: keep WOAD as a verified baseline for recorded sensor data; build a new, phone-camera perception path with a detector selected by measured recall; add camera-motion-compensated tracking so that movement into the walking path can be announced without false alarms from the user's own walking; add offline speech, commands and routing; and measure everything at the application's real frame rate. Review 3 reports the implementation and its results.

The lesson for future use of published assistive systems is that reported numbers should be re-measured on available data before a design is built on them. In this case, measurement changed the direction of the project.

REFERENCES

1. Y. Gao, D. Wu, J. Song, X. Zhang, B. Hou, H. Liu, J. Liao, and L. Zhou, "A wearable obstacle avoidance device for visually impaired individuals with cross-modal learning," *Nature Communications*, vol. 16, Art. no. 2857, Mar. 2025, doi: 10.1038/s41467-025-58085-x.
2. P. Christodoulou, "Soft actor-critic for discrete action settings," arXiv:1910.07207, 2019.
3. G. Jocher *et al.*, "Ultralytics YOLOv5," software, 2020. [Online]. Available: https://github.com/ultralytics/yolov5
4. A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017.
5. T.-Y. Lin *et al.*, "Microsoft COCO: common objects in context," in *Proc. ECCV*, 2014.
6. W. Liu *et al.*, "SSD: single shot multibox detector," in *Proc. ECCV*, 2016.
7. A. Howard *et al.*, "Searching for MobileNetV3," in *Proc. IEEE ICCV*, 2019.
8. Ultralytics, "YOLO26 documentation," software, AGPL-3.0. [Online]. Available: https://docs.ultralytics.com/
9. Roboflow, "RF-DETR: real-time detection transformer," software, Apache-2.0. [Online]. Available: https://github.com/roboflow/rf-detr
10. N. Carion *et al.*, "End-to-end object detection with transformers," in *Proc. ECCV*, 2020.
11. J. Shi and C. Tomasi, "Good features to track," in *Proc. IEEE CVPR*, 1994.
12. B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *Proc. IJCAI*, 1981.
13. J.-Y. Bouguet, "Pyramidal implementation of the Lucas Kanade feature tracker," Intel Corp., Microprocessor Research Labs, Tech. Rep., 2000.
14. M. A. Fischler and R. C. Bolles, "Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography," *Commun. ACM*, vol. 24, no. 6, pp. 381–395, 1981.
15. Y. Zhang *et al.*, "ByteTrack: multi-object tracking by associating every detection box," in *Proc. ECCV*, 2022.
16. A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proc. IEEE ICIP*, 2016.
17. Piper text-to-speech, software. [Online]. Available: https://github.com/rhasspy/piper
18. J. Kim, J. Kong, and J. Son, "Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech," in *Proc. ICML*, 2021.
19. D. Povey *et al.*, "The Kaldi speech recognition toolkit," in *Proc. IEEE ASRU Workshop*, 2011; Vosk offline speech recognition. [Online]. Available: https://alphacephei.com/vosk/
20. P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
21. OpenStreetMap contributors, OpenStreetMap (ODbL 1.0). [Online]. Available: https://www.openstreetmap.org